

OMI-Lisp Native ASCII and JSON Canvas Projection

Normative Structural Diagram Specification

****Version:**** 0.1.0

****Status:**** Canonical implementation draft

****Depends on:**** OMI-Lisp Portable Runtime, ordered CONS serialization, JSON Canvas 1.0 projection profile

****Authority boundary:**** diagrams project structure; diagrams do not validate or accept state

1. Core Principle

OMI-Lisp **MUST** maintain one canonical graph structure and **MAY** render it through multiple surfaces.

```text

canonical OMI object graph

|

+-- OMI-Lisp dotted notation

|

+-- ASCII box diagram

|

+-- JSON Canvas document

|

+-- graphical host canvas

...

The representations **MUST** preserve the same:

```text

node identity

node type

node position

node dimensions

node color

node content

edge identity

source node

destination node

source side

destination side

source endpoint

destination endpoint

edge color

edge label

```

Canonical law:

```
```text
The graph is the structure.

OMI-Lisp is the executable notation.

ASCII is the terminal projection.

JSON Canvas is the interoperable graphical projection.
```
```

---

## # 2. Authority Boundary

```
```text
Parsing constructs structure.

Evaluation computes structure.

Rendering displays structure.

Projection does not validate structure.

Validation determines acceptance.

Receipt records acceptance.
```
```

Changing:

```
```text
ASCII border style

arrow glyph

line routing

canvas zoom

font

screen color theme
```
```

MUST NOT alter the logical graph digest.

Changing:

```
```text
node ID

node type

edge direction

edge endpoint mode

CAR/CDR relation

node payload
```
```

MUST alter the structural identity.

---

### # 3. Native OMI-Lisp Canvas Form

The canonical high-level form is:

```
``lisp
(canvas
 (version . "1.0")
 (nodes . (...))
 (edges . (...))
,,
```

Example:

```
``lisp
(canvas
 (version . "1.0")

 (nodes .
 ((node
 (id . "rose")
 (type . text)
 (x . 0)
 (y . 80)
 (width . 12)
 (height . 3)
 (color . "3")
 (text . "rose"))

 (node
 (id . "violet")
 (type . text)
 (x . 24)
 (y . 80)
 (width . 12)
 (height . 3)
 (color . "6")
 (text . "violet"))

 (node
 (id . "buttercup")
 (type . text)
 (x . 48)
 (y . 80)
 (width . 14)
 (height . 3)
 (color . "2")
 (text . "buttercup")))))

 (edges .
 ((edge
 (id . "rose-to-violet")
 (from-node . "rose")
 (from-side . right)
 (from-end . none)
 (to-node . "violet")
 (to-side . left)
 (to-end . arrow))

 (edge
 (id . "violet-to-buttercup")
 (from-node . "violet")
 (from-side . right)
 (from-end . none)
```

```

 (to-node . "buttercup")
 (to-side . left)
 (to-end . arrow))))))
...

```

```

```

## # 4. Native Node Forms

Every node MUST have:

```

```text
id
type
x
y
width
height
```

```

Optional common field:

```

```text
color
```

```

### ## 4.1 Text node

```

```lisp
(node
  (id . "n1")
  (type . text)
  (x . 0)
  (y . 0)
  (width . 40)
  (height . 8)
  (color . "4")
  (text . "# Heading\n\nMarkdown text"))
```

```

### ## 4.2 File node

```

```lisp
(node
  (id . "n2")
  (type . file)
  (x . 60)
  (y . 0)
  (width . 40)
  (height . 8)
  (file . "docs/model.md")
  (subpath . "#centroid"))
```

```

### ## 4.3 Link node

```

```lisp
(node
  (id . "n3")
  (type . link)
  (x . 120)
  (y . 0)
  (width . 40)
  (height . 8)
  (url . "https://example.invalid"))
```

```

```
...
```

#### ## 4.4 Group node

```
```lisp
(node
  (id . "g1")
  (type . group)
  (x . -10)
  (y . -10)
  (width . 190)
  (height . 100)
  (label . "OMI QuQuart")
  (background . "assets/grid.png")
  (background-style . repeat))
```
```

```

```

#### # 5. Native Edge Form

Every edge MUST have:

```
```text
id
from-node
to-node
```
```

Optional fields:

```
```text
from-side
from-end
to-side
to-end
color
label
```
```

Canonical form:

```
```lisp
(edge
  (id . "e1")
  (from-node . "source")
  (from-side . right)
  (from-end . none)
  (to-node . "notation")
  (to-side . left)
  (to-end . arrow)
  (color . "4")
  (label . "declares"))
```
```

Allowed sides:

```
```text
top
right
bottom
left
```
```

Allowed endpoint forms:

```
```text
none
arrow
```
```

The OMI parser MUST reject unknown canonical side and endpoint names unless an extension profile is explicitly declared.

---

## # 6. Exact JSON Canvas Mapping

The encoder maps:

| OMI-Lisp                   | JSON Canvas                    |
|----------------------------|--------------------------------|
| -----                      | -----                          |
| `from-node`                | `fromNode`                     |
| `from-side`                | `fromSide`                     |
| `from-end`                 | `fromEnd`                      |
| `to-node`                  | `toNode`                       |
| `to-side`                  | `toSide`                       |
| `to-end`                   | `toEnd`                        |
| `background-style`         | `backgroundStyle`              |
| all other canonical fields | same spelling where compatible |

Example encoded edge:

```
```json
{
  "id": "e1",
  "fromNode": "source",
  "fromSide": "right",
  "fromEnd": "none",
  "toNode": "notation",
  "toSide": "left",
  "toEnd": "arrow",
  "color": "4",
  "label": "declares"
},
```
```

The JSON encoder MUST emit both top-level arrays:

```
```json
{
  "nodes": [],
  "edges": []
},
```
```

Empty arrays SHOULD be emitted for deterministic serialization even though the external format permits omission.

---

## # 7. JSON Canvas Field Preservation

The decoder MUST preserve all recognized fields.

The runtime SHOULD also preserve unrecognized extension fields in an opaque property map:

```
```lisp
(extensions .
  (("vendor.example.route" . "orthogonal"))
```
```

```
...("vendor.example.priority" . 3)))
```

Unknown fields MUST NOT be silently converted into executable OMI capabilities.

They are projection metadata unless a registered extension profile defines otherwise.

---

## # 8. Canvas Coordinate Profiles

JSON Canvas positions and sizes use integer pixel coordinates.

ASCII terminals use character cells.

OMI MUST preserve both coordinate spaces separately.

```
``c
typedef struct omi_canvas_metrics {
 int32_t pixel_x;
 int32_t pixel_y;
 uint32_t pixel_width;
 uint32_t pixel_height;

 int32_t cell_x;
 int32_t cell_y;
 uint32_t cell_width;
 uint32_t cell_height;
} omi_canvas_metrics;
```

Conversion requires a named layout profile:

```
``text
json-pixel
ascii-cell
unicode-terminal
braille-terminal
host-native
```

Recommended default conversion:

```
``text
cell_x = floor(pixel_x / horizontal_scale)
cell_y = floor(pixel_y / vertical_scale)
cell_width = max(1, ceil(pixel_width / horizontal_scale))
cell_height = max(1, ceil(pixel_height / vertical_scale))
```

The scaling factors MUST be recorded in the projection context.

---

## # 9. ASCII Rendering Modes

The runtime SHOULD support:

```
``text
ascii
characters limited to 0x20..0x7E

unicode
box-drawing and arrow glyphs permitted
```

```
compact
minimal labels and edges

full
node metadata and edge labels

cons
CAR/CDR cell rendering

canvas
general JSON Canvas rendering
\`\`\`
```

Canonical portable mode:

```
\`\`\`text
ascii
\`\`\`
```

Optional display mode:

```
\`\`\`text
unicode
\`\`\`
```

Receipt identity MUST use canonical structure, not rendered bytes, unless explicitly creating a projection receipt.

---

## # 10. Native CONS Box Renderer

A CONS cell is rendered as two adjacent boxes:

```
\`\`\`text
+-----+-----+
| CAR | CDR |
+-----+-----+
\`\`\`
```

A proper list:

```
\`\`\`lisp
(rose violet buttercup)
\`\`\`
```

may render as:

```
\`\`\`text
+-----+-----+ +-----+-----+ +-----+-----+
| CAR | CDR |--->| CAR | CDR |--->| CAR | CDR |
| rose | o | | violet | o | | buttercup | NIL |
+-----+-----+ +-----+-----+ +-----+-----+
\`\`\`
```

The same structure in strict slot notation:

```
\`\`\`text
+-----+-----+ +-----+-----+ +-----+-----+
| CAR | CDR |--->| CAR | CDR |--->| CAR | CDR |
+-----+-----+ +-----+-----+ +-----+-----+
v			v		v	v
rose			violet		buttercup	NIL
\`\`\`
```

---

## # 11. Empty List and NIL

OMI-Lisp MUST preserve the Lisp equivalence:

```
```text
empty list = NIL
```
```

But it MUST distinguish:

```
```text
bare NIL value
```
```

from:

```
```lisp
(NIL . NIL)
```
```

The diagram:

```
```lisp
(A NIL)
```
```

renders:

```
```text
+-----+-----+      +-----+-----+
| CAR | CDR |---->| CAR | CDR |
|  A  |  o  |      | NIL | NIL |
+-----+-----+      +-----+-----+
```
```

A list terminator does not allocate an additional CONS cell.

Therefore the canonical structure of `(A NIL)` is:

```
```lisp
(A . (NIL . NIL))
```
```

where the final `NIL` is the CDR terminator.

---

## # 12. Nested List Rendering

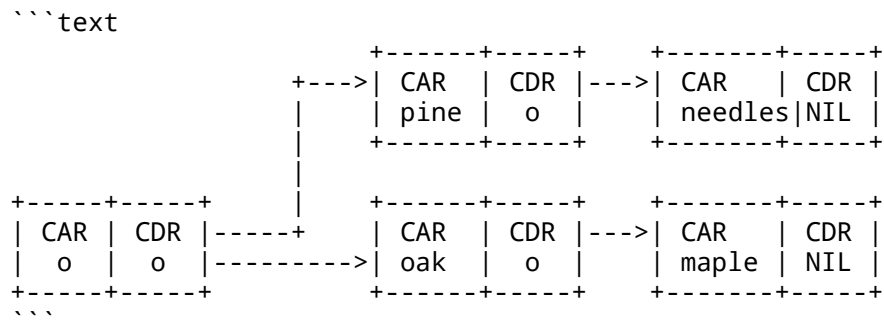
Input:

```
```lisp
((pine needles) oak maple)
```
```

Canonical pair form:

```
```lisp
((pine . (needles . NIL))
 .
 (oak . (maple . NIL)))
```
```

ASCII projection:



The renderer MAY route lines differently, but all references MUST remain correct.

---

### # 13. CONS-to-Canvas Compilation

Every CONS cell MAY compile into one JSON Canvas text node:

```
```lisp
(node
  (id . "cons/0001")
  (type . text)
  (x . 0)
  (y . 0)
  (width . 220)
  (height . 100)
  (text . "| CAR | CDR |\n|---|---|\n| rose | -> cons/0002 |"))
```
```

The CAR reference and CDR reference become edges.

Recommended edge IDs:

```
```text
cons/0001/car
cons/0001/cdr
```
```

Recommended side assignment:

```
```text
CAR reference:
fromSide = bottom
```

```
CDR reference:
fromSide = right
```
```

Destination side:

```
```text
atom target:
toSide = top

next CONS:
toSide = left
```
```

---



## # 15. Edge Direction Semantics

A directed OMI reference uses:

```
```text
fromEnd = none
toEnd = arrow
```
```

A symmetric relation uses:

```
```text
fromEnd = none
toEnd = none
```
```

A reverse arrow uses:

```
```text
fromEnd = arrow
toEnd = none
```
```

A double-headed edge uses:

```
```text
fromEnd = arrow
toEnd = arrow
```
```

ASCII forms:

```
```text
none -> arrow
----->
```
```

```
none -> none

```

```
arrow -> none
<-----
```

```
arrow -> arrow
<----->
```
```

16. Side-Aware ASCII Routing

The renderer MUST honor requested sides when a route is possible.

Node attachment points:

```
```text
top
horizontal center of top border

right
vertical center of right border

bottom
horizontal center of bottom border

left
```

vertical center of left border  
\\\

Routing priority:

```
```text
1. direct straight route
2. one-bend orthogonal route
3. two-bend orthogonal route
4. collision-avoiding routed path
5. labeled fallback route
\\`
```

The renderer MUST NOT silently reverse an edge to make routing easier.

17. Coordinate Collision Policy

If two nodes occupy overlapping ASCII cells, the renderer MUST choose a declared policy:

```
```text
error
clip
layer
shift
auto-layout
\\`
```

Default:

```
```text
layer
\\`
```

Node order determines z-order for projection.

The renderer SHOULD preserve the JSON node-array order.

18. Color Projection

Canonical color forms:

```
```text
hex color
"#RRGGBB"

preset
"1".."6"
\\`
```

ASCII terminals without color support MUST preserve color as metadata.

Example:

```
```text
[COLOR=4]
```
```

ANSI terminals MAY map colors to terminal escape sequences.

ANSI output is noncanonical display output.

---

## # 19. Text and Markdown

Text nodes contain Markdown-capable strings.

The ASCII renderer MAY:

```
```text
render Markdown literally

render a simplified terminal interpretation

strip formatting while preserving text

show a compact preview
```
```

It MUST NOT change the underlying stored text.

Recommended modes:

```
```text
literal
preview
plain
```
```

---

## # 20. Groups

A group node is a visual container.

ASCII rendering:

```
```text
+=====+
| GROUP: OMI QuQuart |
|
|   +-----+   +-----+
|   | Q0 SOURCE |----->| Q1 NOTATE |
|   +-----+   +-----+
|
+=====+
```
```

Containment is derived geometrically from node rectangles unless an OMI extension records explicit membership.

Group containment MUST NOT imply semantic ownership or authority.

---

## # 21. QuQuart Canvas Example

```
```lisp
```

```

(canvas
  (version . "1.0")

  (nodes .
    ((node
      (id . "q0")
      (type . text)
      (x . 0)
      (y . 10)
      (width . 24)
      (height . 5)
      (color . "3")
      (text . "Q0\nSOURCE\nRULES + FACTS")))

    (node
      (id . "q1")
      (type . text)
      (x . 36)
      (y . 10)
      (width . 24)
      (height . 5)
      (color . "4")
      (text . "Q1\nNOTATION")))

    (node
      (id . "q2")
      (type . text)
      (x . 72)
      (y . 10)
      (width . 24)
      (height . 5)
      (color . "5")
      (text . "Q2\nREADING\nCOMBINATORS")))

    (node
      (id . "q3")
      (type . text)
      (x . 108)
      (y . 10)
      (width . 24)
      (height . 5)
      (color . "6")
      (text . "Q3\nRECEIPT\nCLOSURES")))

    (node
      (id . "cons")
      (type . text)
      (x . 54)
      (y . 28)
      (width . 24)
      (height . 5)
      (color . "1")
      (text . "CONS\nCENTROID")))))

  (edges .
    ((edge
      (id . "q0-q1")
      (from-node . "q0")
      (from-side . right)
      (to-node . "q1")
      (to-side . left)
      (to-end . arrow))

    (edge
      (id . "q1-q2")

```

```

    (from-node . "q1")
    (from-side . right)
    (to-node . "q2")
    (to-side . left)
    (to-end . arrow))

(edge
  (id . "q2-q3")
  (from-node . "q2")
  (from-side . right)
  (to-node . "q3")
  (to-side . left)
  (to-end . arrow))

(edge
  (id . "q0-cons")
  (from-node . "q0")
  (from-side . bottom)
  (to-node . "cons")
  (to-side . left)
  (to-end . arrow))

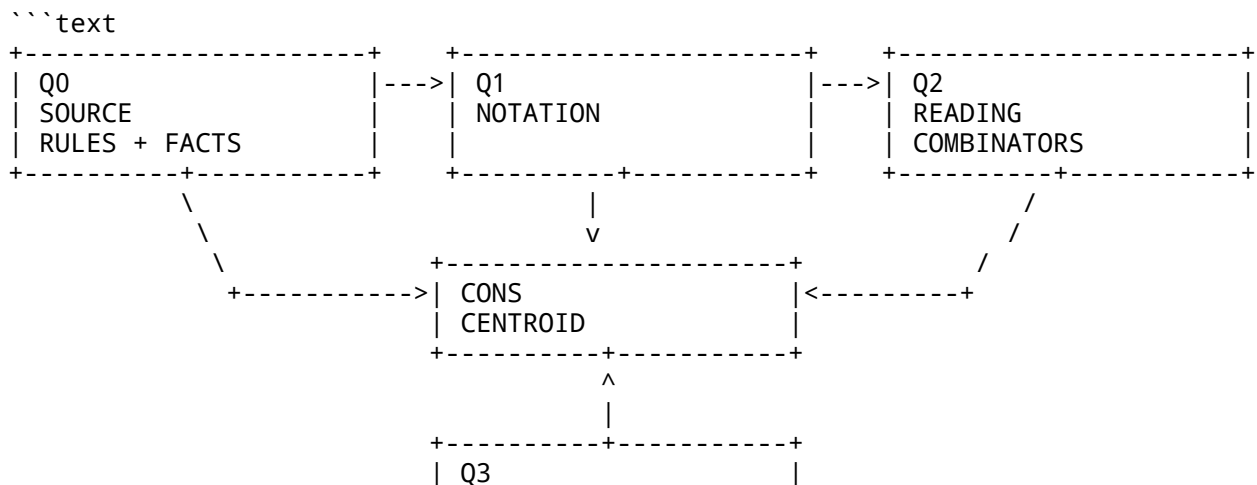
(edge
  (id . "q1-cons")
  (from-node . "q1")
  (from-side . bottom)
  (to-node . "cons")
  (to-side . top)
  (to-end . arrow))

(edge
  (id . "q2-cons")
  (from-node . "q2")
  (from-side . bottom)
  (to-node . "cons")
  (to-side . top)
  (to-end . arrow))

(edge
  (id . "q3-cons")
  (from-node . "q3")
  (from-side . bottom)
  (to-node . "cons")
  (to-side . right)
  (to-end . arrow))))
...

```

Possible ASCII projection:



	RECEIPT	
	CLOSURES	
	+-----+	

...

22. Truth-Function Canvas Integration

The sixteen rank-2 Boolean functions may be represented natively as:

```
```lisp
(truth-canvas
 (rank . 2)
 (layout . gray)
 (node-template . wittgenstein)
 (edges .
 (gray-adjacency
 complement
 narrative)))
...

```

This expands into an ordinary OMI canvas graph.

No special JSON Canvas extension is required for the visual structure.

OMI-specific truth metadata remains inside:

```
```text
text content

stable node IDs

optional OMI extension fields
```

```

---

## # 23. Round-Trip Requirements

The runtime MUST support:

```
```text
OMI-Lisp canvas
-> internal canvas AST
-> JSON Canvas
-> internal canvas AST
-> OMI-Lisp canvas
```

```

The following MUST survive:

```
```text
IDs

types

coordinates

dimensions

recognized node-specific fields

colors

```

edge direction
edge sides
edge endpoints
labels
extension fields
```

ASCII rendering is not required to round-trip visually.

The canonical internal AST and JSON/OMI encodings are round-trippable.

---

## # 24. Canonical Ordering

For deterministic output:

```text  
nodes
preserve declared z-order

edges
preserve declared order

object fields
emit in canonical profile order

extension keys
sort lexicographically
```

The canonical digest MUST bind node-array order because node order carries display stacking semantics.

---

## # 25. Native Built-ins

Recommended procedures:

```lisp  
(canvas nodes edges)

(canvas-node fields...)

(canvas-edge fields...)

(canvas-add-node canvas node)
(canvas-add-edge canvas edge)

(canvas-remove-node canvas id)
(canvas-remove-edge canvas id)

(canvas-find-node canvas id)

(canvas-validate canvas)

(canvas->json canvas)

```
(json->canvas json)
(canvas->ascii canvas)
(cons->canvas value)
(cons->ascii value)
```\
```

Projection options:

```
``lisp
(canvas->ascii canvas
 (mode . full)
 (charset . ascii)
 (routing . orthogonal)
 (color . metadata))
```\
```

26. Validation Rules

Canvas validation MUST check:

```
``text
every node has a unique nonempty ID
every edge has a unique nonempty ID
every node type is recognized
coordinates are valid integers
width and height are positive integers
every edge source exists
every edge destination exists
side names are valid
endpoint names are valid
type-specific required fields exist
color syntax is valid
group background style is valid
```\
```

A structurally valid canvas is not necessarily an accepted OMI interpretation.

Canvas validation and semantic validation are separate.

---

## # 27. Error Examples

Invalid:

```
``lisp
(edge
 (from-node . "a")
 (to-node . "b"))
```\
```

```
...
```

Reason:

```
```text
missing edge ID
```
```

Invalid:

```
```lisp
(node
 (id . "a")
 (type . text)
 (x . 0)
 (y . 0)
 (width . 20)
 (height . 5))
```
```

Reason:

```
```text
text node missing text
```
```

Invalid:

```
```lisp
(edge
 (id . "e")
 (from-node . "missing")
 (to-node . "a"))
```
```

Reason:

```
```text
source node does not exist
```
```

```
---
```

28. C Rendering API

```
```c
typedef struct omi_ascii_render_options {
 uint32_t columns;
 uint32_t rows;

 bool use_unicode;
 bool show_colors;
 bool show_edge_labels;
 bool show_node_ids;
 bool auto_layout;

 uint8_t routing_profile;
} omi_ascii_render_options;

omi_status omi_canvas_parse(
 omi_runtime *runtime,
 omi_value form,
 omi_canvas **out,
 omi_error *error
);
```

```

omi_status omi_canvas_validate(
 const omi_canvas *canvas,
 omi_error *error
);

omi_status omi_canvas_write_json(
 const omi_canvas *canvas,
 omi_writer *writer,
 omi_error *error
);

omi_status omi_canvas_read_json(
 omi_runtime *runtime,
 omi_slice json,
 omi_canvas **out,
 omi_error *error
);

omi_status omi_canvas_render_ascii(
 const omi_canvas *canvas,
 const omi_ascii_render_options *options,
 omi_writer *writer,
 omi_error *error
);

omi_status omi_cons_render_ascii(
 omi_runtime *runtime,
 omi_value value,
 const omi_ascii_render_options *options,
 omi_writer *writer,
 omi_error *error
);

```

---

## # 29. Implementation Modules

```

```text
include/omi/canvas.h
include/omi/ascii.h
include/omi/json_canvas.h
include/omi/layout.h
include/omi/routing.h

```

```

src/canvas.c
src/canvas_validate.c
src/ascii_render.c
src/ascii_route.c
src/cons_diagram.c
src/json_canvas_encode.c
src/json_canvas_decode.c
src/layout.c
```

```

Tests:

```

```text
tests/canvas_parse_test.c
tests/canvas_validation_test.c
tests/json_canvas_roundtrip_test.c
tests/ascii_render_test.c
tests/cons_diagram_test.c
tests/edge_routing_test.c

```

...

30. Required Conformance Tests

```text

simple three-element proper list

list containing NIL

nested list

improper pair

shared tail

shared atom

all four edge sides

all four side-to-side combinations

none/none endpoint

none/arrow endpoint

arrow/none endpoint

arrow/arrow endpoint

text node

file node

link node

group node

hex color

six preset colors

node z-order preservation

JSON round trip

OMI-Lisp round trip

ASCII structure snapshot

```

For structural tests:

```text

changing only rendering width  
does not change logical CONS identity

changing CAR target  
does change logical CONS identity

changing CDR target  
does change logical CONS identity

reversing an edge

does change graph identity

moving a node  
changes projection identity  
but need not change semantic model identity  
```\n

31. Identity Profiles

The implementation SHOULD expose two digests.

Semantic digest

Binds:

```\ntext  
node IDs  
  
node types  
  
node payloads  
  
edge identities  
  
edge direction  
  
edge labels when semantic  
  
OMI relations  
```\n

May exclude:

```\ntext  
x  
y  
width  
height  
display color  
```\n

Projection digest

Binds the entire JSON Canvas-compatible structure:

```\ntext  
coordinates  
  
dimensions  
  
colors  
  
z-order  
  
all edge-side choices  
  
all endpoint styles  
```\n

A receipt MUST state which digest profile it uses.

32. Final Canon

```
```text
OMI-Lisp can describe a canvas natively.

The canvas remains executable dotted structure.
```
```

```
```text
The same canvas can render as:
```

```
OMI-Lisp
ASCII
JSON Canvas
or a graphical host view.
```
```

```
```text
CAR and CDR references become directed edges.

Atoms and nested CONS cells become nodes.
```
```

```
```text
top
right
bottom
left

are preserved as edge attachment semantics.
```
```

```
```text
none
arrow

are preserved independently
at the source and destination endpoints.
```
```

```
```text
Node type,
position,
size,
color,
content,
grouping,
and z-order

remain projection data.
```
```

```
```text
ASCII diagrams are native projections
of machine-readable OMI structure.
```

```
They are not hand-maintained pictures.
```
```

```
```text
JSON Canvas is the interoperable canvas carrier.

OMI-Lisp is the executable structural notation.

The internal canvas AST is the shared model.
```

...

```text

Projection does not accept.

Validation accepts.

Receipt records.

```